LAB3: FPGA 응용 회로

실험 전 보고서

학과: 전자전기컴퓨터공학부

이름/학번: 정현재/2023440127

작성일: 2026/09/27

대상: LED PWM/RGB PWM/피에조/스텝 모터/MMSS 시계/문자 LCD/UART Echo

검증 환경: VS Code VaporView

보고서 및 실험의 범위

LED PWM, RGB PWM, 피에조, 스텝 모터, MMSS 시계, 문자 LCD, UART Echo의 총 7개 실험을 대상으로 Verilog HDL 설계와 사전 시뮬레이션을 정리했다. 실제 보드 구현·계측은 수업에서 수행할 항목으로 구분했다.

소스 식별

프로젝트: lab3

기준 커밋: 6b3b06b434656f3209f5eb2e263d71f399239235

실행본: src/, sim/, constraints/

GitHub 주소: https://github.com/Jeong719/ECE2_Lab3.git

제출 태그: v1.0

1. 목적과 공통 검증 조건

실험 목적

조합논리회로와 순차논리회로에 대해서 배운 점을 기반으로 총 7종의 FPGA 응용 회로를 Verilog HDL로 설계하고, 시뮬레이션을 통해 예상한 결과와 일치하는지 검증한다.

관련 이론

PWM: 디지털 신호의 HIGH와 LOW 비율을 빠르게 조절하여 아날로그 전압처럼 기기를 제어하는 기술

스텝 모터의 동작 원리: 스텝 모터는 코일에 전류를 공급하는 순서에 따라 회전자가 일정한 각도씩 이동하는 모터이다. 코일의 여자 상태를 순차적으로 변경하면 자기장의 방향이 바뀌고, 회전자가 이를 따라 움직인다. 여자 순서를 반대로 변경하면 회전 방향이 바뀌며, 단위 시간당 스텝 수를 조절하면 회전 속도를 조절할 수 있다. 따라서 스텝 모터를 제어하기 위해서는 코일 구동 패턴의 순서와 패턴 변경 간격을 결정해야 한다.[1]

검증 환경과 실행 방법

사전 검증은 VS Code에서 Icarus Verilog를 이용하여 수행하고, 생성된 VCD 파일을 VaporView로 열어 파형을 분석한다.

통과 기준과 파형 분석 기준

정상 및 복구 실행은 테스트벤치의 모든 검사를 통과하여 LAB3_PASS가 공통적으로 출력되는 것을 통과 기준으로 한다. 실행 로그의 검사 수와 종료 시각을 기록하고 전체 파형과 각 실험마다 주요 기능이 담긴 구간의 확대 파형을 통해 조건에 따라 올바르게 출력됐는지 분석한다.

시간 단위와 검증 범위

파형의 시간 단위는 ns를 기준으로 해석하며, ps로 표시된 경우 1000ps를 1ns로 환산한다. TB의 10ns 클록은 기능 검증을 위한 조건이다.

2. 실험1: LED PWM

설계 파일: lab3_led_pwm.v, button_onepulse.v, pwm_channel.v

설계와 예상 결과

본 회로는 PWM 신호의 듀티비를 조절하여 LED의 밝기를 변경하는 회로다. 이 때 한 주기는 매우 짧기 때문에 사람의 눈에는 점등과 소등이 아닌 밝기의 변화로 인식하게 된다. 버튼 입력을 통해 밝기 단계 level은 0부터 10까지 증가하며, 10에서 버튼 입력이 이루어지면 0으로 다시 순환하는 구조다.

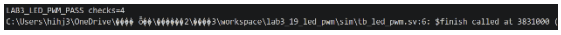
Level이 증가할수록 LED가 켜져 있는 시간의 비율이 증가할 것으로 예상된다.

RTL 파일의 핵심 코드는 다음과 같다.

```
always @(posedge clk_50mhz or
posedge rst_p) begin
    if (rst_p)
        level <= 4'd0;
    else if (press)
        level <= (level == LEVELS) ?
4'd0 : level + 1'b1;
end
```

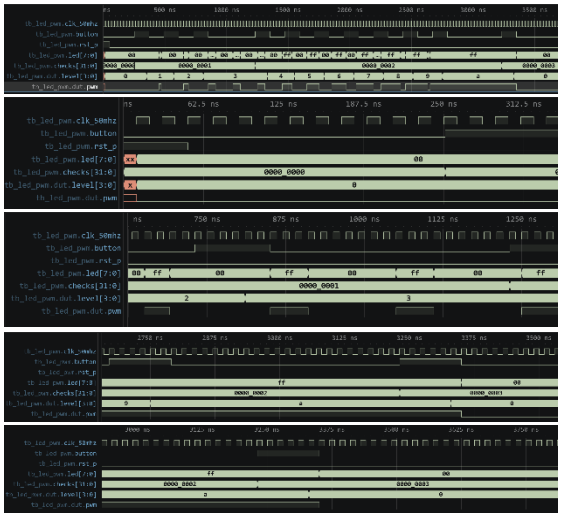
이 코드에서 press를 통해 level이 한 단계 증가하는 기능을 구현하였다.

사전 검증과 파형 해석



사전 검증 결과 총 4회 검사, 3831ns에 끝난 것을 확인할 수 있다.

아래는 실험1의 파형과 주목해야 할 부분이다.



2번째 사진에서 level=0일 때 LED 출력은 늘 Low, 즉 00을 유지하는 것을 확인할 수 있다.

3번째 사진에서 level=3일 때, 한 pwm 주기(10 clk 주기)동안 LED 출력이 ff인 시간의 비율이 30%인 것을 확인할 수 있다.

4번째 사진에서 level=10일 때 LED 출력은 항상 High, 즉 ff를 유지하는 것을 확인할 수 있다.

5번째 사진에서 level이 10에서 0으로 변한 이후 다음 상승 에지에서 LED 출력이 ff에서 00으로 변하는 것을 확인할 수 있다.

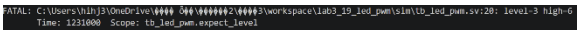
src > lab3_led_pwm.v

```
1  `timescale 1ns/1ps
2  module lab3_led_pwm #(
3      parameter integer CLK_HZ = 50_000_000,
4      parameter integer PWM_HZ = 1_000,
5      parameter integer LEVELS = 5,
6      parameter integer DEBOUNCE_CYCLES = 1_000_000
7  ) (
8      input  wire clk_50mhz,
9      input  wire rst_p,
10     input  wire button,
11     output wire [7:0] led
```

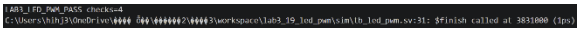
sim > tb_led_pwm.sv

```
1  `timescale 1ns/1ps
2  module tb_led_pwm;
3      reg clk_50mhz=0, rst_p=1, button=0;
4      wire [7:0] led;
5      integer checks=0, high_count, i;
6      always #10 clk_50mhz=~clk_50mhz;
7      lab3_led_pwm #(.CLK_HZ(1000),.PWM_HZ(100),.LEVELS(5),.DEBOUNCE_CYCLES(2)) dut
8          (.clk_50mhz(clk_50mhz),.rst_p(rst_p),.button(button),.led(led));
9
```

위의 사진은 소스파일 및 테스트벤치 파일에서 LEVELS를 10에서 5로 변경한 코드의 사진이다.



해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 해당 코드로 진행했을 때 버튼 3회 후 HIGH가 기댓값인 3클록 대신 6클록이 되었기 때문이다.



이후 다시 LEVELS를 10으로 변경했을 때 정상적으로 PASS가 뜨는 것을 확인할 수 있다.

보드에서 확인할 것

밝기 조절 버튼을 누를 때마다 LED의 밝기가 단계적으로 증가하는지 확인한다. 또한, 버튼을 길게 눌러도 한 단계만 증가하며, 최대 밝기에서 버튼을 한 번 더 누르면 LED가 꺼지는지, 리셋 버튼을 눌렀을 때 LED가 꺼지는지 확인한다.

3. 실험2: RGB PWM

설계 파일: lab3_rgb_pwm.v, button_onepulse.v, pwm_channel.v

설계와 예상 결과

본 회로는 LED_PWM을 활용하여 빨강, 초록, 파랑 LED에 각각 독립적인 PWM 신호를 공급하여 각각의 밝기를 조절하여 혼합색을 조절하는 회로다. 각 색의 밝기는 level_r, level_g, level_b에 저장되며 각 값은 0부터 10까지 증가한 뒤 0으로 순환한다.

RTL 파일의 핵심코드는 다음과 같다.

```
if (press_r)
    level_r <= (level_r == LEVELS) ? 0 :
```

```
level_r + 1'b1;
if (press_g)
    level_g <= (level_g == LEVELS) ? 0 :
level_g + 1'b1;
if (press_b)
    level_b <= (level_b == LEVELS) ? 0 :
level_b + 1'b1;

assign led_r = {4{pwm_r}};
assign led_g = {4{pwm_g}};
assign led_b = {4{pwm_b}};
```

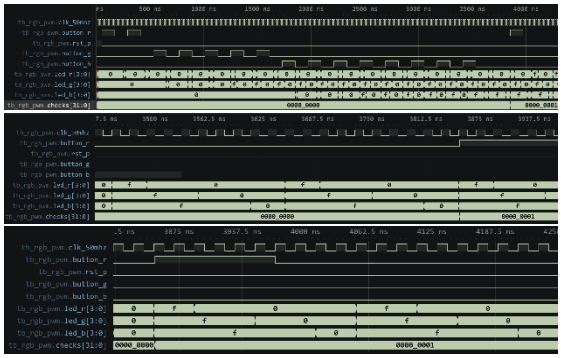
이 코드를 통해 각 색상의 밝기 단계를 독립적으로 변경할 수 있다.

사전 검증과 파형 해석



사전 검증 결과 총 2회 검사 및 4431ns가 소요된 것을 확인할 수 있다.

아래는 실험 2의 파형과 주목해야 할 부분에 대한 사진이다.



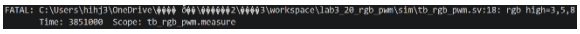
2번째 사진에서 R=20%, G=50%, B=80%인 구간에서 한 pwm 주기 안에서 led_r은 총 2클록 주기동안 high 상태를 유지하고, led_g는 총 5클록 주기 동안 high 상태를 유지하고, led_b는 총 8클록 주기동안 high 상태를 유지하는 것을 확인할 수 있다.

3번째 사진에서 r 버튼만 누르고 g 버튼 및 b 버튼을 누르지 않았을 때 level_r이 1 증가하는 것을 확인할 수 있다.



```
 7  ) (
12      input  wire button_b,
13      output wire [3:0] led_r,
14      output wire [3:0] led_g,
15      output wire [3:0] led_b
16  );
17      wire press_r, press_g, press_b;
18      wire pwm_r, pwm_g, pwm_b;
19      reg [3:0] level_r, level_g, level_b;
20
21      button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_r (
22          .clk(clk_50mhz), .rst_p(rst_p), .button(button_r), .pulse(press_r));
23      button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_g (
24          .clk(clk_50mhz), .rst_p(rst_p), .button(button_g), .pulse(press_g));
25      button_onepulse #(.STABLE_CYCLES(DEBOUNCE_CYCLES)) u_button_b (
26          .clk(clk_50mhz), .rst_p(rst_p), .button(button_b), .pulse(press_b));
27
28      always @(posedge clk_50mhz or posedge rst_p) begin
29          if (rst_p) begin
30              level_r <= 1; level_g <= 0; level_b <= 0;
31          end else begin
32              if (press_r) level_r <= (level_r == LEVELS) ? 0 : level_r + 1'b1;
33              if (press_g) level_g <= (level_g == LEVELS) ? 0 : level_g + 1'b1;
34              if (press_b) level_b <= (level_b == LEVELS) ? 0 : level_b + 1'b1;
35          end
36      end
37
```

위의 사진은 소스 파일에서 rst_p 이후 level_r이 0이 아닌 1로 초기화되게끔 변경한 사진이다.



해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 위에서 설명했듯이 첫 검사에서 RGB의 HIGH 클록 수가 기댓값 2,5,8 대신 3,5,8이 되었기 때문에 실패한 것이다.



이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

RGB의 각 밝기를 조절할 때마다 혼합색이 변하는지 확인한다. 또한, 각 색에 해당하는 버튼을 눌렀을 때 해당 색상의 밝기만 변경되는지 확인한다.

4. 실험3: 피에조

설계 파일: lab3_piezo.v

설계와 예상 결과

본 회로는 일정한 간격으로 출력을 반전시켜 피에조에 사각파를 공급하고 음을 발생시키는 회로다. 이 때 카운터를 통해 피에조에 공급되는 사각파의 주파수를 변경하

고 변경된 주파수는 음의 높낮이를 변경한다.

이를 수행하는 핵심 코드는 다음과 같다.

```
localparam integer HALF_PERIOD =
CLK_HZ / (2 * TONE_HZ);

if (count == HALF_PERIOD - 1) begin
    count <= 0;
    piezo <= ~piezo;
end else begin
    count <= count + 1'b1;
end
```

이 코드를 통해 목표 주파수의 반 주기에 해당하는 클록 수를 센 후 출력을 반전한다. 두 번 반전하면 한 주기가 완성되므로 TONE_HZ에 가까운 주파수의 사각파가 생성된다.

사전 검증과 파형 해석



사전 검증 결과 총 5회의 엣지 및 551ns가 소요된 것을 확인할 수 있다.

아래는 실험 3의 파형이다.

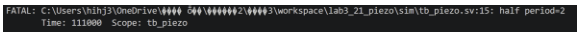


Piezo가 한 번 반전될 때까지 clk 5주기가 소요된 것을 확인할 수 있다.

```
src > lab3_piezo.v
1   `timescale 1ns/1ps
2   module lab3_piezo #(
3       parameter integer CLK_HZ = 50_000_000,
4       parameter integer TONE_HZ = 440
5   ) (
6       input  wire clk_50mhz,
7       input  wire rst_p,
8       output reg  piezo
9   );
10      localparam integer HALF_PERIOD = CLK_HZ / (2 * TONE_HZ);
11      localparam integer COUNT_WIDTH = (HALF_PERIOD < 2) ? 1 : $clog2(HALF_PERIOD
12      reg [COUNT_WIDTH-1:0] count;
13
14      always @(posedge clk_50mhz or posedge rst_p) begin
15          if (rst_p) begin
16              count <= 0;
17              piezo <= 0;
18          end else if (count == HALF_PERIOD - 1) begin
19              count <= 0;
20              piezo <= ~piezo;
21          end else begin
22              count <= count + 1'b1;
23          end
24      end
25  endmodule
```

```
src > tb_piezo.v
1   `timescale 1ns/1ps
2   module tb_piezo;
3       reg clk_50mhz=0,rst_p=1;
4       wire piezo;
5       integer cycles,last_cycle=-1,edges=0;
6       always #10 clk_50mhz=~clk_50mhz;
7       lab3_piezo #(.CLK_HZ(1000),.TONE_HZ(200)) dut(.clk_50mhz(clk_50mhz),.rst_p(rst_p),.piezo(piezo) );
8       initial begin
9         $dumpfile("wave.vcd");$dumpvars(0,tb_piezo);
10        repeat(3)@(posedge clk_50mhz);rst_p=0;
11        for(cycles=1;cycles<=25;cycles=cycles+1)begin
12          @(posedge clk_50mhz);#1;
13          if(piezo!==dut.piezo)$fatal(1,"unknown output");
14          if(dut.count==0)begin
15            if(last_cycle>0 && cycles-last_cycle!=5)$fatal(1,"half period=%0d",cycles-last_cycle);
16            last_cycle=cycles;edges=edges+1;
17          end
18        end
19        if(edges<4)$fatal(1,"too few edges");
20        $display("LAB3_PIEZO_PASS edges=%0d",edges);$finish;
21      end
22      initial begin #5000;$fatal(1,"timeout");end
23  endmodule
```

위의 사진은 소스 파일 및 테스트벤치 파일에서 TONE_HZ를 294->440, 100->200으로 변경한 코드의 사진이다.



해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 테스트벤치에서 반전 간격이 정수 나눗셈에 따라 2클록으로 바뀌어 실패했기 때문이다.



이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

피에조에서 설정한 주파수에 해당하는 소리가 지속적으로 발생하는지 확인한다 주파수를 높였을 때 높은 음이, 낮췄을 때 낮은 음이 발생하는지 확인한다.

5. 실험4: 스텝 모터

설계 파일: lab3_stepper.v

설계와 예상 결과

스텝 모터는 전기 신호를 정해진 순서로 주면 일정한 각도씩 회전하는 모터다. 본 회로는 이 순서를 나타내는 제어 신호를 만들고 모터 구동 회로가 코일에 필요한 전류를 공급하게끔 하는 회로다.

총 4가지의 상태를 생성하여 상태가 변할 때마다 모터가 한 스텝씩 이동하도록 구동

한다.

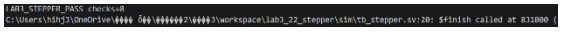
이를 수행하는 핵심 코드는 다음과 같다.

```
state <= direction_sync ? state - 1'b1 :
state + 1'b1;


case (state)
    2'd0:     stepmotor = 4'b0011;
    2'd1:     stepmotor = 4'b0110;
    2'd2:     stepmotor = 4'b1100;
    default: stepmotor = 4'b1001;
endcase
```

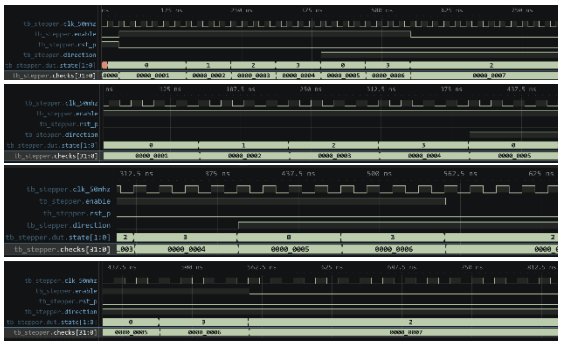
이 코드를 통해 방향 입력에 따라 상태를 증가시키거나 감소시켜 코일 구동 순서를 변경한다. 출력이 현재 상태에 의해 결정되므로 무어 머신 구조로 설명할 수 있다.

사전 검증과 파형 해석



사전 검증 결과 총 8회 검사 및 831ns가 소요된 것을 확인할 수 있다.

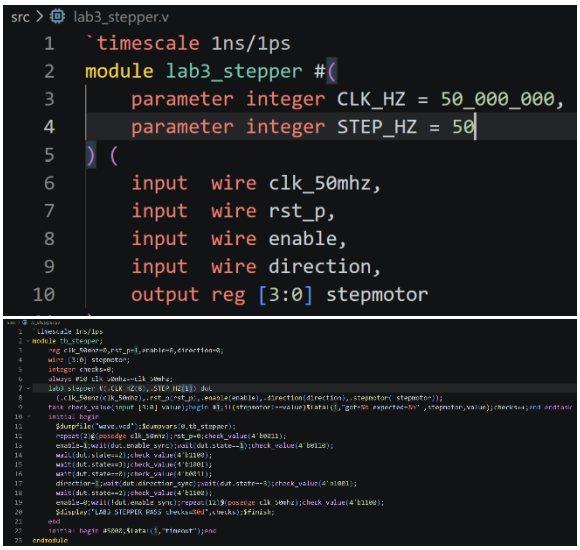
아래는 실험 4의 파형과 주목해야 할 부분에 대한 사진이다.



2번째 사진에서 0->1->2->3으로 정방향 상태 변화 주기를 관찰할 수 있다.

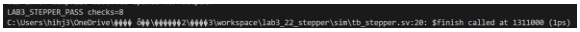
3번째 사진에서 정방향으로 변화하다가 direction이 1로 변한 이후, 역방향으로 상태가 변화하는 것을 확인할 수 있다.

이후 4번째 사진에서 enable=0이 되자, 시간이 아무리 지나도 상태가 변화하지 않는 것을 볼 수 있다.

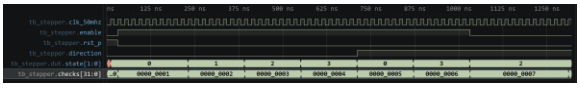


```
`timescale 1ns/1ps
module lab3_stepper #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer STEP_HZ = 50
) (
    input  wire clk_50mhz,
    input  wire rst_p,
    input  wire enable,
    input  wire direction,
    output reg [3:0] stepmotor
```


위의 사진은 소스 파일과 테스트벤치 파일에서 단계 속도를 절반으로 변경한 코드의 사진이다.



해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 기존 코드에 비해 소요 시간이 길어졌지만 PASS 문구가 나타나는 것을 확인할 수 있다. 이는 TB의 단계 간격이 증가했지만, TB가 상태가 바뀔 때까지 기다려 패턴을 검사하기 때문이다.



위 사진은 해당 코드로 진행했을 때의 파형이다. 전체적인 모양은 같지만, 각 상태가 유지되는 시간이 두 배로 길어지는 것을 확인할 수 있다.



이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

Direction이 각각 0과 1일 때 모터의 회전 방향이 다른 방향인지 확인한다. 또한, 모

터의 회전 각도가 일정한지 확인한다.

6. 실험5: MMSS 시계

설계 파일: lab3_mmss_clock.v, mmss_counter.v, sevenseg_decode.v

설계와 예상 결과

본 회로는 분과 초를 계산하고 7세그먼트에 표시하는 디지털 시계다. Mmss_counter 모듈은 입력 클록을 세어 1초 간격을 생성하며, 분의 십의 자리와 일의 자리, 초의 십의 자리와 일의 자리를 각각 BCD 값으로 저장한다.

초가 59에서 증가하면 00으로 초기화되고 분이 증가하며, 분도 59까지 증가한 뒤 순환할 것으로 예상된다.

핵심 코드는 다음과 같다.

```
if (subsecond == CLK_HZ - 1) begin
    subsecond <= 0;
    if (second_ones != 9)
        second_ones <= second_ones + 1'b1;
    else begin
        second_ones <= 0;
        if (second_tens != 5)
            second_tens <= second_tens + 1'b1;
end
end else begin
    subsecond <= subsecond + 1'b1;
end
```

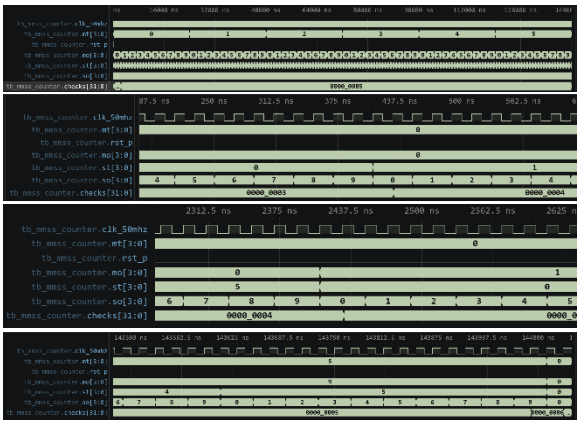
이 코드를 통해 기준 클록을 세어 1초마다 일의 자리를 증가시키고 일의 자리가 9면 0으로 돌아가며 십의 자리로 자리 올림한다.

사전 검증과 파형 해석



사전 검증 결과 총 7회 검사 및 144031ns가 소요된 것을 확인할 수 있다.

아래는 실험 5의 파형과 주목해야 할 부분에 대한 사진이다.



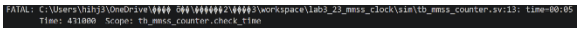
두 번째 사진에서 so가 9까지 증가한 이후 0으로 변화하는 것을 통해 순환하는 구조임을 확인할 수 있고, 동시에 st가 1로 변하는 것을 통해 자리 올림 현상을 확인할 수 있다.

세 번째 사진에서 st=5, so=9 이후 st=0, so=0으로 변하며 mo=1이 되는 것을 통해 00:59초 이후 01:00이 되는 것을 확인할 수 있다.

네 번째 사진에서 mt=5, mo=9, st=5, so=9 이후 모두 0으로 변하는 것을 통해 59:59 이후 00:00으로 순환하는 구조를 확인할 수 있다.

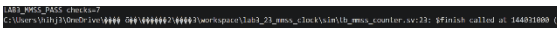


위의 사진은 테스트벤치 파일의 9번째 줄에서 CLK_HZ(2)를 CLK_HZ(4)로 수정한 코드다.



해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이

는 초 증가 간격이 40ns에서 80ns로 증가했기 때문에 실패했기 때문이다.



이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

세그먼트에 표시된 각 자릿수가 순환하는 구조인지 확인한다. 또한 59초 이후에 1:00이 되는지 확인하며 동작 중 리셋 버튼을 누르면 00:00으로 초기화되는지 확인한다.

7. 실험6: 문자 LCD

설계 파일: lab3_character_lcd.v

설계와 예상 결과

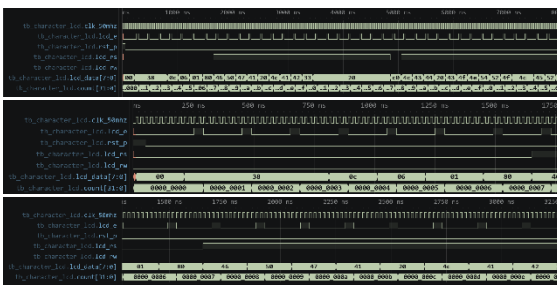
본 회로는 LCD에 명령과 문자 데이터를 정해진 순서와 타이밍으로 전달하여 글자를 표시하는 회로다. Lcd_rs=0이면 명령, 1이면 데이터를 뜻한다. 명령은 LCD의 동작과 표시 위치를 정하고 문자 데이터는 표시되는 문자를 정한다.

사전 검증과 파형 해석



사전 검증 결과 총 40바이트 및 8130ns가 소요된 것을 확인할 수 있다.

아래는 실험 6의 파형과 주목해야 할 부분에 대한 사진이다.

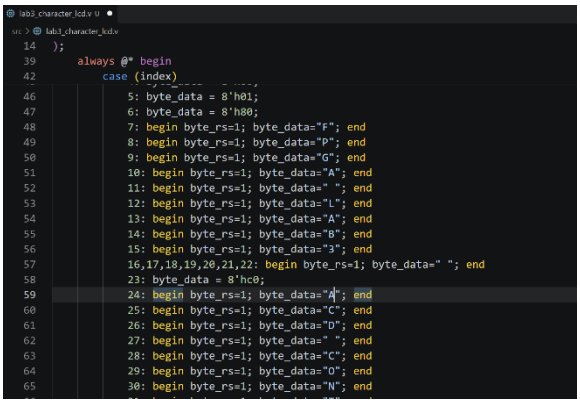




두 번째 사진에서 rs=0, 명령은 38, 0c, 06, 01이다. 이들은 초기화 명령으로 문자를 보내기 전 LCD의 동작 방식과 화면 상태를 설정하는 명령이다.

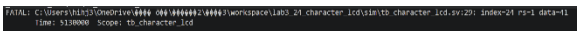
세 번째 사진에서 Rs=0일 때 첫번째 줄에 대한 주소 명령 80 이후 RS=1로 바꾸고 문자 데이터를 전송한 것을 확인할 수 있다. 이 때, 각 문자 데이터는 F,P,G,A,공백,L,A,B,3를 뜻한다.

네 번째 사진에서 rs=0일 때 두 번째 줄에 대한 주소 명령 c0 이후 rs=1로 바꾸고 문자 데이터를 전송한 것을 확인할 수 있다. 이 때, 각 문자 데이터는 L,C,D,공백,C,O,N,T,R,O,L,L,E,R,공백을 뜻한다.



```
lab3_character_lcd.v U
src > lab3_character_lcd.v
14  );
39      always @* begin
42          case (index)
46              5: byte_data = 8'h01;
47              6: byte_data = 8'h80;
48              7: begin byte_rs=1; byte_data="F"; end
49              8: begin byte_rs=1; byte_data="P"; end
50              9: begin byte_rs=1; byte_data="G"; end
51              10: begin byte_rs=1; byte_data="A"; end
52              11: begin byte_rs=1; byte_data=" "; end
53              12: begin byte_rs=1; byte_data="L"; end
54              13: begin byte_rs=1; byte_data="A"; end
55              14: begin byte_rs=1; byte_data="B"; end
56              15: begin byte_rs=1; byte_data="3"; end
57              16,17,18,19,20,21,22: begin byte_rs=1; byte_data=" "; end
58              23: byte_data = 8'hc0;
59              24: begin byte_rs=1; byte_data="A"; end
60              25: begin byte_rs=1; byte_data="C"; end
61              26: begin byte_rs=1; byte_data="D"; end
62              27: begin byte_rs=1; byte_data=" "; end
63              28: begin byte_rs=1; byte_data="C"; end
64              29: begin byte_rs=1; byte_data="O"; end
65              30: begin byte_rs=1; byte_data="N"; end
```

위의 사진은 소스 파일의 59번째 줄에서 "L"을 "A"로 바꾼 코드의 사진이다.



해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. TB의 기댓값 L 대신 A가 전송되어 실패했기 때문이다.



이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

전원을 켜거나 리셋한 후 LCD가 초기화되는지 확인한다. 또한 명령 및 데이터에 대한 조작이 잘 작동하는지, 줄바꿈 명령을 실행한 후 데이터를 입력시켰을 때 정상적으로 줄이 바뀐 곳에 문자가 표시되는지 확인한다.

8. 실험7: UART Echo

설계 파일: lab3_uart_echo.v, uart.v

설계와 예상 결과

본 회로는 PC에서 UART로 수신한 문자를 같은 값으로 다시 송신하는 에코 회로다. RX는 수신 신호, TX는 송신 신호다.

핵심 코드는 다음과 같다.

```
tx_valid <= 0;
if (rx_valid) begin
    last_data <= rx_data;
    if (tx_ready) begin
        tx_data <= rx_data;
        tx_valid <= 1;
    end
end

assign led = last_data;
```

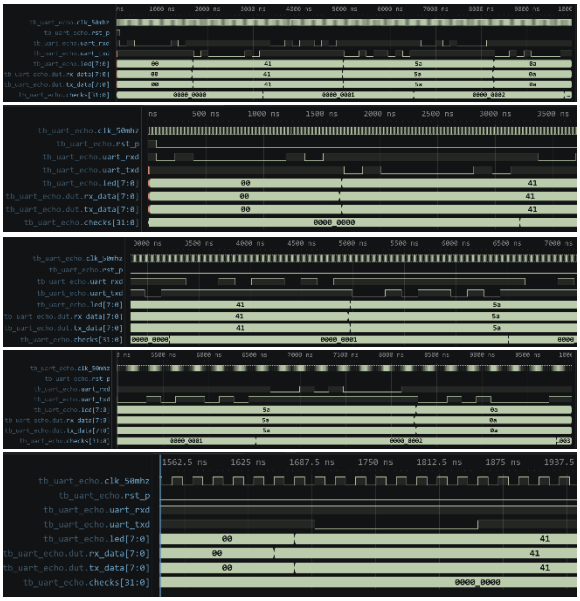
이 코드를 통해 수신이 완료되면 데이터를 저장하여 LED에 표시한다. 송신 회로가 준비되어 있으면 같은 데이터를 전달하고 tx_valid를 활성화하여 echo 전송을 요청한다.

사전 검증과 파형 해석



사전 검증 결과 총 3회 검사 및 9970ns가 소요된 것을 확인할 수 있다.

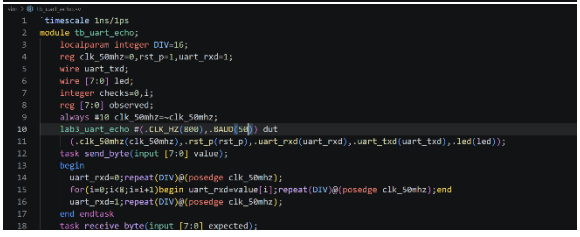
아래는 실험 7의 파형과 주목해야 할 부분에 대한 사진이다.



두번째 사진에서 rx_data값이 41인 이후 tx_data 값도 41 즉 A를 수신 후 A(0x41)를 송신하는 것을 확인할 수 있다.
세 번째 사진에서도 마찬가지로 rx 값이 5a인 직후 tx 값도 5a, 즉 Z를 수신 후 Z(0x5a)를 송신하는 것을 확인할 수 있다.
네 번째 사진에서도 마찬가지로 0a의 에코를 확인할 수 있다.
다섯 번째 사진에 rx_data가 변한 이후 다음 상승 에지에서 led 및 tx_data가 rx_data와 같은 값으로 변하는 것을 확인할 수 있다.

src > lab3_uart_echo.v

```
1   `timescale 1ns/1ps
2   module lab3_uart_echo #(
3       parameter integer CLK_HZ = 50_000_000,
4       parameter integer BAUD = 4_800
5   ) (
6       input  wire clk_50mhz,
7       input  wire rst_p,
8       input  wire uart_rxd,
9       output wire uart_txd,
10      output wire [7:0] led
11  );
```

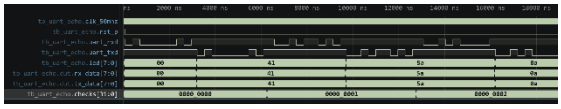


```
1   `timescale 1ns/1ps
2   module tb_uart_echo;
3       localparam integer DIV=16;
4       reg clk_50mhz=0,rst_p=1,uart_rxd=1;
5       wire uart_txd;
6       wire [7:0] led;
7       integer checks=0,i;
8       reg [7:0] observed;
9       always #10 clk_50mhz=~clk_50mhz;
10      lab3_uart_echo #(.CLK_HZ(800),.BAUD(50)) dut
11        (.clk_50mhz(clk_50mhz),.rst_p(rst_p),.uart_rxd(uart_rxd),.uart_txd(uart_txd),.led(led));
12      task send_byte(input [7:0] value);
13      begin
14        uart_rxd=0;repeat(DIV)@(posedge clk_50mhz);
15        for(i=0;i<8;i=i+1)begin uart_rxd=value[i];repeat(DIV)@(posedge clk_50mhz);end
16        uart_rxd=1;repeat(DIV)@(posedge clk_50mhz);
17      end endtask
18      task receive_byte(input [7:0] expected);
```

위의 사진은 소스파일의 4번째 줄에서 BAUD를 9600에서 4800, 테스트벤치 파일의 3, 10번째 줄에서 각각 DIV를 8에서 16, BAUD를 100에서 50으로 바꾼 코드의 사진이다.



해당 코드로 진행하면 터미널에 위와 같은 PASS문구가 나타나는 것을 확인할 수 있다. 이는 TB의 비트 시간이 증가하고, 송수신 속도가 서로 맞기 때문이다.



위 사진은 해당 코드로 진행했을 때의 파형이다. 기존 파형에 비해서 비트 폭이 두 배로 길어지고 수신 완료 및 에코 송신 시점이 늦어지는 것을 확인할 수 있다.



이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

보드에 문자를 전송했을 때 동일한 문자가 돌아오는지 확인한다. 또한 수신한 마지막 문자의 ASCII 값이 LED에 이진수로 표시되는지 확인한다.

9. 실험 수행 계획

1단계(프로젝트 생성 및 코딩)

Vivado를 실행하여 실습 대상 회로의 RTL 프로젝트를 생성하고 Design Sources에 Verilog 모듈 코드 작성

2단계(테스트벤치 구동 및 기능 검증)

Simulation Sources에 테스트벤치를 추가하고 Run Behavioral Simulation을 실행하여 Vivado 내장 시뮬레이터로 출력파형 확인

3단계(결과 파형 분석 및 검증)

Vivado Waveform 창에서 입력 벡터에 따른 출력값의 변화를 확인하고, 이론적 결과와 일치하는지 대조하여 시뮬레이션 통과 확인

4단계(보드 구현)

Vivado 2026.1을 통해 fpga 보드와 연결 후 실제 보드에서 조작을 통해 해당 회로의 주요 기능을 구현

10. 참고 문헌

[1] M. Morris Mano and Michael D. Ciletti, *Digital Design, Global Edition*, 6th ed., Pearson, 2018, Chapters 5–6, 8.